

Unpaid tier and Google One users: Gemini CLI was replaced by Antigravity CLI on June 18th, 2026. To learn more, see our [blog post](#).



Gemini CLI

[Home](#)[Plans](#)[► Extensions](#)[Docs](#)[Reference](#)[Resources](#)

Agent Skill best practices



Create high-quality, reliable Agent Skills by following these established design principles and patterns.

Design for discovery

The most important part of a skill is its description. This is the only information the model has before activation.

- **Be specific:** Use keywords that are likely to appear in user prompts (for example, “audit,” “security,” “refactor,” “migration”).
- **Define the trigger:** Clearly state *when* the skill should be used (for example, “Use this skill when the user asks to review a PR for performance regressions”).
- **Avoid overlap:** Ensure your skill descriptions are distinct from one another and from the general capabilities of the model.

Progressive disclosure

The “context window” is a shared resource. Use a three-level loading system to manage context efficiently.

1. **Metadata (name + description):** Always in context (~100 words).
2. **SKILL.md body:** Loaded only after the skill triggers (<5k words).
3. **Bundled resources:** Loaded only as needed by the model.

This website uses [cookies](#) from Google to deliver and enhance the quality of its services and to analyze traffic.

[I understand.](#)

Best practice: Keep the `SKILL.md` body focused on core procedural instructions. Move detailed reference material, schemas, and examples into separate files in a `references/` directory.

Degrees of freedom

Match the level of instruction specificity to the task's fragility.

- **High freedom (text-based instructions):** Use when multiple approaches are valid or decisions depend heavily on context.
- **Medium freedom (pseudocode or scripts with parameters):** Use when a preferred pattern exists but some variation is acceptable.
- **Low freedom (specific scripts, few parameters):** Use when operations are fragile and error-prone, or a specific sequence **MUST** be followed.

Bundle resources effectively

Leverage the skill's ability to include scripts and assets to extend the agent's capabilities.

- **Use scripts for deterministic tasks:** If a task can be automated with a script (for example, running a linter, fetching data from an API), bundle it in the `scripts/` folder.
- **Agentic ergonomics:** Ensure scripts output LLM-friendly stdout. Suppress verbose tracebacks and provide clear, concise success/failure messages.
- **Provide templates:** Include common file headers or boilerplate code in the `assets/` folder to ensure the agent produces consistent output.

Anatomy of a great skill

A well-structured skill directory organizes its resources into specialized sub-folders.

my-skill/

SKILL.md

scripts/

assets/

(Required) Core instructions and metadata

(Optional) Executable logic (Node.js, Python, etc.)

(Optional) Templates and non-executable resources

This website uses cookies from Google to deliver and enhance the quality of its services and to analyze its usage.

Security and privacy

Design your skills with security in mind to protect your workspace and data.

- **Avoid hardcoded secrets:** Never include API keys or passwords in your skill's scripts or documentation.
- **Review third-party skills:** Inspect the SKILL.md and scripts of any skill before installing it from an untrusted source.
- **Limit scope:** Design skills to be as focused as possible to minimize the potential impact of errors.

📅 Last updated: Apr 30, 2026

Google for Developers

Terms | Privacy

This website uses [cookies](#) from Google to deliver and enhance the quality of its services and to analyze traffic.